

Documentação de Solução Técnica: Arquitetura Bitemporal do pipeline para cálculo do GMV

Autor: Aíla Lima **Case:** Data Engineer - Teachable

1. Visão Geral

Este documento detalha o design e a arquitetura do pipeline de dados para o cálculo de **Gross Merchandise Value (GMV)**. Mais do que agregar valores financeiros, o desafio central deste case consiste em lidar com eventos de origem (CDC) que chegam de forma independente, atrasada e fora de ordem, estando sujeitos a correções contínuas.

Para garantir que reprocessamentos e chegadas tardias não reescrevam a "verdade histórica", a solução implementada é uma **tabela fato bitemporal e append-only**. Essa abordagem separa estritamente o tempo do negócio do tempo do sistema, garantindo reprodutibilidade, auditoria e resiliência.

2. Contexto

O processamento de streams de CDC para tabelas financeiras gera uma complexidade analítica: precisamos responder a perguntas diferentes sem corromper o estado dos dados.

- **Tempo de Negócio:** "Qual foi o GMV real gerado em janeiro de 2023?"
- **Tempo de Sistema:** "Qual era a nossa visão (o que acreditávamos ser o GMV) de janeiro de 2023 quando fechamos o mês no dia 31?"

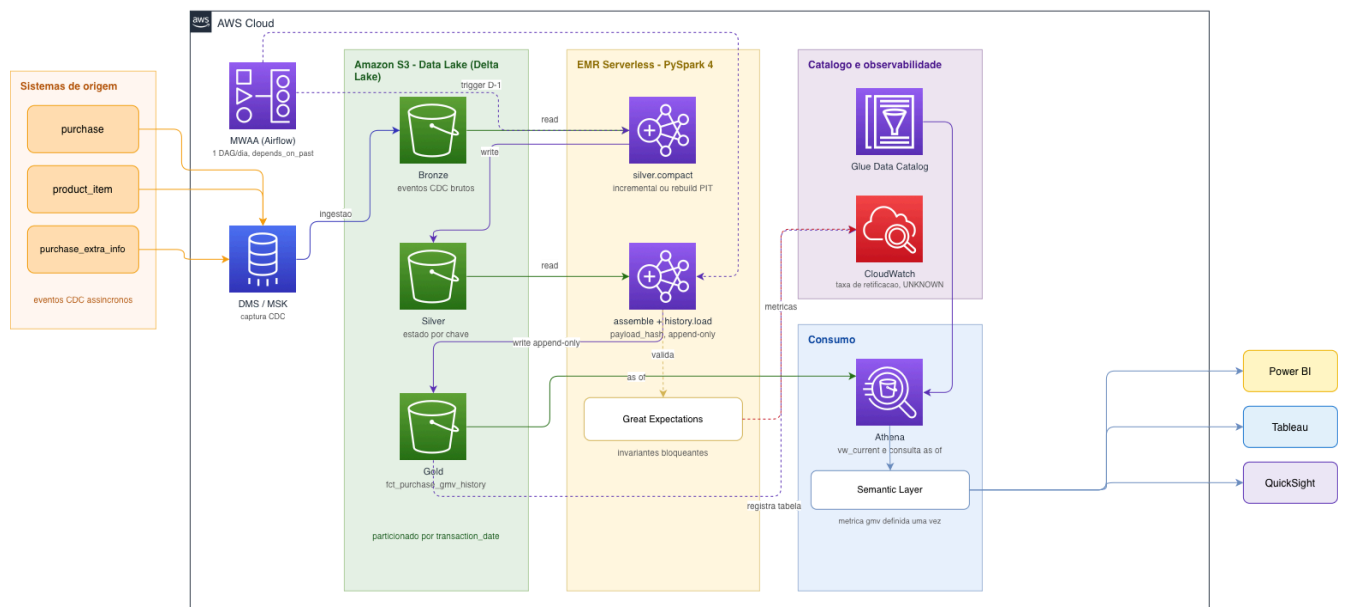
Para atender à necessidade de auditoria e consultas point-in-time ("as of"), o modelo expõe ambos os eixos:

Eixo	Colunas	Pergunta que responde
Tempo de negócio (valid time)	order_date, release_date, gmV_date	quando aconteceu
Tempo de sistema (transaction time)	transaction_date, version_valid_from_ts	quando ficamos sabendo

3. Arquitetura de Dados (Medalhão)

A solução adota a arquitetura Medallion para garantir um fluxo seguro e com rastreabilidade total:

- **Camada Bronze:** Funciona como landing zone do CDC bruto, particionada por dia de ingestão. É imutável e atua como o registro definitivo do que foi recebido.
- **Camada Silver:** Atua como um cache mutável. Armazena apenas a versão mais recente observada de cada compra. Pode ser completamente destruída e reconstruída a partir da Bronze a qualquer momento.
- **Camada Gold:** A fonte única da verdade (`fct_purchase_gmv_history`). É estritamente append-only e versionada.
- **Camada Serving:** Views descartáveis para consumo dos usuários de negócios e BI



4. Stack Tecnológico e Justificativas

Considerando a escala de dados e a complexidade de agregações (window functions, joins complexos), as seguintes ferramentas foram definidas:

- **Processamento - PySpark 4:** Ferramenta nativa do paradigma Lakehouse, ideal para o processamento stateful entre as camadas.
- **Armazenamento em Delta Lake:** Escolhido pelas garantias de `delta.appendOnly`, commits atômicos e suporte nativo a operações de **MERGE** para os upserts na camada Silver. (O pipeline também roda em formato parquet puro como fallback).
- **Orquestração com Apache Airflow:** Configurado com `depends_on_past=True` para garantir a correte cronológica dos batches diários.

5. Decisões Críticas de Engenharia (Trade-offs)

5.1. Tabela Fato Append-Only (Sem `valid_to`)

Diferente de implementações SCD2 tradicionais que atualizam uma coluna `valid_to` ou `is_current`, esta modelagem **nunca fecha uma versão**. Fechar uma versão exigiria modificar uma partição do passado, violando a regra de imutabilidade.

- **Vantagem Analítica:** As consultas de viagem no tempo ("as of") tornam-se operações altamente eficientes. Basta adicionar um predicado na chave de partição (`WHERE transaction_date <= 'Data'`), transformando o que seria um *full scan* em um rápido *partition pruning*.

5.2. Hash de Payload para Controle de Versão

Versões são identificadas usando um hash gerado a partir dos campos de negócio (`payload_hash`). Se um evento de CDC for reenviado de forma idêntica, o hash não muda, evitando a criação de novas versões desnecessárias e impedindo que a tabela infle com ruídos operacionais.

5.3. Fuso Horário Rigorosamente Controlado (UTC)

Um risco crítico identificado no desenvolvimento foi o uso do fuso horário do sistema operacional pelo **PySpark** durante a conversão de *datetimes*. Em ambientes UTC-3, eventos do fim do dia eram empurrados para a partição do dia seguinte, falsificando consultas históricas. A solução injeta um timezone UTC forçado a nível de sistema e emite timestamps com *tzinfo* explícito.

5.4. Reconstrução Point-in-Time da Camada Silver

O reprocessamento de dados passados (backfills) exige cuidado para que eventos do futuro não vazem para cálculos do passado. A camada Silver possui um mecanismo que, ao recalcular dias antigos, ignora dados ingeridos após a data do backfill, garantindo reprodutibilidade determinística em qualquer reexecução.

6. Premissas de Negócio Assumidas

Durante o mapeamento de requisitos, algumas ambiguidades nos dados de origem precisaram ser tratadas de forma assertiva:

- **Definição do dia do GMV:** Assumimos `gmv_date` = `release_date`, pois o valor só é de fato gerado na captura bem-sucedida. O `order_date` foi mantido para análises sob outra perspectiva.
- **Exclusão de Custos:** A coluna `order_transaction_cost_hist` não entra no cálculo de GMV, por se tratar de um custo dedutível que afeta a receita líquida/take rate, não o valor transacionado bruto.
- **Eventos Faltantes (Unknowns):** Se uma compra chega, mas seus componentes auxiliares (ex: *subsidiary*) estão atrasados, inserimos um registro transitório (`UNKNOWN`). É melhor ter uma linha sinalizada do que ocultar a transação.

7. Qualidade de Dados (Observabilidade)

A estratégia de qualidade foi separada em duas frentes distintas para evitar fadiga de alertas:

- **Testes CI/CD (Regressão de Código):** Executados a cada commit. Testam regras de negócio e versionamento bitemporal usando cenários determinísticos simulados, com foco especial em garantir que o reprocessamento não altere partições antigas.
- **Testes de Produção (Data Quality):** Atuam como *Quality Gates* no momento da execução do batch.
 - **Bloqueantes:** Abortam o pipeline se houver quebra de grão único, valores de GMV negativos ou versões inválidas.
 - **Alertas (Warnings):** Não param a esteira, mas notificam sobre atrasos além do esperado (ex: compras presas como UNKNOWN há 7 dias) ou volumes anômalos de retificação.

8. Tópicos avançados (bônus)

8.1. Evolução para tempo real

O design atual em batch (D-1) atende aos requisitos de conciliação financeira, estabelecendo uma fronteira auditável limpa. No entanto, a arquitetura foi desenhada adotando o padrão **Kappa**. Caso haja demanda de negócio para visões intradiárias, a migração para **Real-time (Streaming)** exigiria apenas trocar o batch da camada Silver por um `flatMapGroupsWithState` no *Structured Streaming*, sem alterar a semântica da tabela Gold ou quebrar as consultas históricas existentes.

8.2. Camada semântica de métricas

A Gold é a tabela fato, não a métrica. `gmv` é definido uma única vez em dbt Semantic Layer como `SUM(gmv_amount)` sobre `vw_purchase_gmv_current`, com `gmv_date` como dimensão temporal principal e `order_date` como alternativa. O parâmetro `as_of` vira uma dimensão de primeira classe, e é isso que permite a um usuário de BI puxar "janeiro como reportado em março" sem escrever window function.

8.3. Conciliação com Finance

GMV é bruto e próximo de regime de caixa; receita é líquida e por competência. A ponte:

None

GMV (bruto, em release_date)

- reembolsos e chargebacks ← version_type = RESTATEMENT, STATUS_CHANGED
- VAT ← order_transaction_cost_hist
- custos de parcelamento ← order_transaction_cost_hist
- = receita líquida

O grão bitemporal é o que torna isso auditável: para qualquer período contábil encerrado, o snapshot "as of" da data de fechamento é reproduzível para sempre, e todo movimento posterior carrega um `change_reason` que o explica. O lançamento original nunca é editado, um novo é feito.